

ITA0614 - MACHINE LEARNING

ASSIGNMENT CO2-AT3

1. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

Problem

A decision tree performs very well on the training data but poorly on unseen (test) data. This indicates **overfitting**, where the model memorizes the training data instead of learning general patterns.

Possible Causes

1. Overfitting

- The tree learns every detail of the training data.
- It also learns noise and outliers.
- Training accuracy becomes very high, but testing accuracy decreases.

2. Deep Tree Structure

- Too many levels (depth).
- Many branches with very few samples.
- Complex rules reduce generalization.

3. Noisy Data

- Incorrect labels.
- Missing values.
- Duplicate records.
- Outliers affecting split decisions.

4. Irrelevant Features

- Unnecessary features confuse the model.
- Increase model complexity.

5. Small Training Dataset

- Insufficient examples.
- The model cannot learn generalized patterns.

Debugging Steps

Step 1: Compare Training and Testing Accuracy

Example:

- Training Accuracy = **99%**
- Testing Accuracy = **75%**

Large difference indicates overfitting.

Step 2: Visualize the Tree

Check:

- Tree depth
- Number of leaves
- Complex branches

Step 3: Analyze Feature Importance

Remove features with very low importance.

Step 4: Check Data Quality

- Remove duplicates
- Handle missing values
- Correct wrong labels
- Remove outliers

Step 5: Perform Error Analysis

Identify which samples are misclassified and determine whether errors are due to noise or insufficient data.

Optimization Methods

1. Pruning

Pre-Pruning

Stop tree growth early by:

- Limiting maximum depth
- Setting minimum samples per leaf

- Setting minimum samples for splitting

Benefits:

- Simpler model
- Reduced overfitting

Post-Pruning

Grow the full tree first and then remove unnecessary branches.

Benefits:

- Better generalization
- Improved test accuracy

2. Cross-Validation

Use **k-fold cross-validation**.

Example:

- Divide data into 5 folds.
- Train on 4 folds.
- Test on 1 fold.
- Repeat five times.

Benefits:

- Reliable performance estimation.
- Reduces overfitting.

3. Feature Selection

Select only important features.

Methods:

- Information Gain
- Chi-Square Test
- Correlation Analysis
- Recursive Feature Elimination (RFE)

Benefits:

- Less complexity
- Faster training
- Better accuracy

4. Hyperparameter Tuning

Optimize:

- Maximum depth
- Minimum samples split
- Minimum samples leaf
- Criterion (Gini or Entropy)

5. Ensemble Methods

Use:

- Random Forest
- Gradient Boosting

These combine multiple trees to improve accuracy and reduce overfitting.

Outcome

Using pruning, feature selection, cross-validation, and proper hyperparameter tuning improves the decision tree's ability to generalize, resulting in better performance on unseen data.

2. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy.

Problem

A Convolutional Neural Network (CNN) shows unstable accuracy and loss during training, making learning inconsistent.

Possible Causes

1. Improper Data Augmentation

Examples:

- Excessive rotation
- Large zoom
- Incorrect flipping
- Strong brightness changes

Effects:

- Images become unrealistic.
- Model struggles to learn useful patterns.

2. Vanishing Gradients

Occurs in very deep networks.

Symptoms:

- Very slow learning
- Training accuracy remains low
- Loss stops decreasing

Cause:

Gradients become extremely small during backpropagation.

3. Batch Normalization Errors

Problems include:

- Incorrect placement of Batch Normalization layers
- Very small batch size
- Incorrect training/evaluation mode

Effects:

- Unstable learning
- Fluctuating accuracy

4. Learning Rate Too High

Effects:

- Loss oscillates
- Model cannot converge
- Accuracy fluctuates

5. Poor Weight Initialization

Random initialization may:

- Slow convergence
- Cause unstable training

6. Small Dataset

Insufficient images cause:

- Overfitting
- Poor generalization
- High variance in results

7. Class Imbalance

Some classes have many more images than others.

Effects:

- Bias toward majority classes
- Poor performance on minority classes

Debugging Strategies

Step 1: Monitor Learning Curves

Plot:

- Training Loss
- Validation Loss
- Training Accuracy
- Validation Accuracy

Look for sudden spikes or divergence.

Step 2: Check Data Augmentation

Ensure augmentations are realistic and preserve important image features.

Step 3: Verify Batch Normalization

- Place Batch Normalization after convolution layers and before activation (common practice).
- Use appropriate batch sizes.
- Ensure correct training and evaluation modes.

Step 4: Inspect Gradients

Monitor gradient values to detect vanishing or exploding gradients.

Step 5: Experiment with Learning Rate

Use:

- Smaller learning rate
- Learning rate scheduler
- Warm-up strategy

Step 6: Validate the Dataset

Check for:

- Incorrect labels
- Duplicate images
- Corrupted files
- Class imbalance

Optimization Techniques

1. Better Data Augmentation

Use moderate transformations:

- Random crop
- Horizontal flip
- Small rotation
- Brightness adjustment

2. Batch Normalization

Benefits:

- Faster convergence
- Stable gradients
- Higher accuracy

3. Use ReLU Activation

Advantages:

- Reduces vanishing gradients
- Faster training
- Better convergence

4. Dropout

Randomly deactivate neurons during training.

Benefits:

- Prevents overfitting
- Improves generalization

5. Learning Rate Scheduling

Reduce the learning rate gradually during training to improve convergence.

6. Early Stopping

Stop training when validation loss no longer improves.

Benefits:

- Prevents overfitting
- Saves computation time

7. Transfer Learning

Use pre-trained models such as:

- ResNet
- VGG
- MobileNet
- EfficientNet

Benefits:

- Faster convergence
- Better performance with limited data

Outcome

By applying proper data augmentation, correcting batch normalization, using suitable learning rates, monitoring gradients, employing regularization (Dropout), and leveraging transfer learning, CNN training becomes more stable, converges more reliably, and achieves higher accuracy on unseen images.